

استخدام الخوارزمية الجينية لاختيار أفضل الخصائص (Features) من بيانات صور الثدي لتمييز الصور السليمة عن المصابة بالسرطان.

#### الخطوات الأساسية:

1. توليد مجموعة حلول عشوائية. (Population)
  - كل حل يمثل مجموعة مختارة من الخصائص أو معاملات.
2. حساب دالة اللياقة. (Fitness Function)
  - نسبة دقة التصنيف (Accuracy) عند استخدام الخصائص المحددة.
3. اختيار الأفضل. (Selection)
4. تنفيذ عمليات Crossover و Mutation لتوليد حلول جديدة.
5. التكرار حتى تصل لحل أمثل أو بعد عدد أجيال محدد.

#### الأدوات المستخدمة

- برمجة بلغة Python مع مكتبات (deap, sklearn, numpy)
- بيانات سرطان الثدي (Dataset) مجاني (sklearn):  
قاعدة بيانات مكونة من 91 حالة، لكل واحدة 30 خاصية مرتبطة بتشخيص الورم، و خانة تمثل تشخيص كل حالة (مصاب 0 / سليم 1)

#### المشكلة :

نريد اختيار الخصائص الأهم من بيانات سرطان الثدي لتصنيف المرضى.

#### الحل :

استخدمنا الخوارزمية الجينية (GA) لاختيار أفضل مجموعة. Features

#### النتيجة :

GA اختارت 15 خاصية فقط من أصل 30، ومع دقة عالية جدًا 99.4%

#### الفائدة:

- تقليل وقت الحساب (لأن عدد الخصائص أقل).
- تجنب الإفراط في التعلم. (Overfitting)

- تحسين كفاءة النماذج الطبية في التنبؤ.

#### المرحلة القادمة:

#### تحسين الإعدادات (Tuning Parameters)

- تغيير حجم المجتمع (Population size) (مثلاً 100 بدل 30).
- تغيير عدد الأجيال (Generations) (مثلاً 100 بدل 20).
- تغيير معدل التقاطع (Crossover probability).
- تغيير معدل الطفرة (Mutation probability).

الهدف: هل ممكن نحصل على دقة أعلى أو خصائص أقل بعد ضبط الإعدادات.

#### دمج مع خوارزميات أخرى: (Hybrid Approach)

- دمج GA مع خوارزمية Particle Swarm Optimization (PSO) لتحسين البحث.
- دمج GA مع Simulated Annealing يساعد في الهروب من الحلول المحلية.
- دمج GA مع الشبكات العصبية (ANNs) تستعمل GA لاختيار الخصائص و ANN للتصنيف.

#### تحسين دالة اللياقة (Fitness Function)

حالياً دالة اللياقة = الدقة فقط. لكن ممكن تطويرها لتكون:

$$\text{Fitness} = \alpha * \text{Accuracy} - \beta * (\text{عدد الخصائص المستخدمة})$$

الهدف هو الحصول على دقة عالية مع تقليل عدد الخصائص. (Feature Reduction)

مثال: لدينا 30 خاصية، الخوارزمية تختار 10 فقط مع دقة 98% ، أفضل من 15 اختيار خاصية مع دقة 99%.

#### التصنيف باستخدام نماذج أقوى

بدل Logistic Regression يمكن استخدام:

➤ Random Forest

➤ Support Vector Machine (SVM)

#### Cross-Validation بدلاً من Train/Test split

- حالياً تم تقسيم البيانات Train/Test مرة واحدة.
- الأفضل أن نستخدم k-Fold Cross Validation داخل GA أثناء حساب fitness.
- هذا يعطي نتائج أكثر ثباتاً.

### الخلاصة:

المرحلة الأولى كانت تطبيق GA بنجاح، أما المرحلة الثانية فستكون تحسين GA عبر ضبط المعاملات، تطوير دالة اللياقة، أو دمجها مع خوارزميات أخرى.

### الكود العملي:

#تطبيق عملي للخوارزمية الجينية لاختيار الخصائص (Feature Selection)

#على داتاسيت سرطان الثدي (Breast Cancer dataset)

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
import random
import arabic_reshaper
from bidi.algorithm import get_display
```

1. #تحميل البيانات

```
data = load_breast_cancer()
X = data.data
y = data.target
n_features = X.shape[1]
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

2. #تقسيم البيانات (تدريب + اختبار)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

#إعداد الخوارزمية الجينية GA

#دالة لياقة: (Fitness Function) تقيس الدقة باستخدام الخصائص المختارة

```
def fitness(individual):
```

# تحويل الفرد (سلسلة من 1/0) إلى قائمة خصائص

```

selected_features = [i for i in range(n_features) if individual[i] == 1]

# لو ما اختار ولا خاصية
if len(selected_features) == 0:

    return 0,

# تدريب نموذج بسيط (Logistic Regression) على الخصائص المختارة
clf = LogisticRegression(max_iter=5000)
clf.fit(X_train[:, selected_features], y_train)
y_pred = clf.predict(X_test[:, selected_features])
acc = accuracy_score(y_test, y_pred)

return acc,

# إعداد مكتبة DEAP الخاصة بالـ GA
from deap import base, creator, tools

# إنشاء نوع جديد: "كل فرد = قائمة من 1/0 بطول عدد الخصائص"
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
toolbox = base.Toolbox()
toolbox.register("attr_bool", random.randint, 0, 1) # جينات ثنائية (0/1)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool, n=n_features)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# تسجيل العمليات
toolbox.register("evaluate", fitness)
toolbox.register("mate", tools.cxTwoPoint) # تقاطع (Crossover)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05) # طفرة (Mutation)
toolbox.register("select", tools.selTournament, tournsize=3)

# تنفيذ الخوارزمية الجينية
def run_ga():
    pop = toolbox.population(n=30) # حجم المجتمع
    NGEN = 20 # عدد الأجيال
    for gen in range(NGEN):
        offspring = toolbox.select(pop, len(pop))
        offspring = list(map(toolbox.clone, offspring))

```

# تقاطع + طفرة

```
for child1, child2 in zip(offspring[::2], offspring[1::2]):
```

```
    if random.random() < 0.7:
```

```
        toolbox.mate(child1, child2)
```

```
        del child1.fitness.values
```

```
        del child2.fitness.values
```

```
for mutant in offspring:
```

```
    if random.random() < 0.2:
```

```
        toolbox.mutate(mutant)
```

```
        del mutant.fitness.values
```

# تقييم الأفراد

```
invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
```

```
fitnesses = map(toolbox.evaluate, invalid_ind)
```

```
for ind, fit in zip(invalid_ind, fitnesses):
```

```
    ind.fitness.values = fit
```

```
pop[:] = offspring
```

# أفضل فرد في هذا الجيل

```
fits = [ind.fitness.values[0] for ind in pop]
```

```
الجيل    text1= "
```

```
    reshaped_text1 = arabic_resaper.reshape(text1) # correct its shape
```

```
    bidi_text1 = get_display(reshaped_text1) # correct its direction
```

```
أفضل دقة    text2= "
```

```
    reshaped_text2 = arabic_resaper.reshape(text2) # correct its shape
```

```
    bidi_text2 = get_display(reshaped_text2) # correct its direction
```

```
: أفضل مجموعة خصائص مختارة:    text3= "
```

```
    reshaped_text3 = arabic_resaper.reshape(text3) # correct its shape
```

```
    bidi_text3 = get_display(reshaped_text3) # correct its direction
```

```
دقة النموذج النهائي    text4= "
```

```
    reshaped_text4 = arabic_resaper.reshape(text4) # correct its shape
```

```
    bidi_text4 = get_display(reshaped_text4) # correct its direction
```

```
print(f"{bidi_text1} {gen+1}: {bidi_text2} = {max(fits):.4f}")
```

```

best_ind = tools.selBest(pop, 1)[0]

print(f"\n✅ {bidi_text3}")

print([i for i in range(n_features) if best_ind[i] == 1])

print(f"{bidi_text4} = {fitness(best_ind)[0]:.4f}")

```

#تشغيل الخوارزمية

run\_ga()

النتائج:

```

PS C:\Users\Nadia Qwaider\desktop> python breast_cancer_ga.py
الجيل 1: أفضل دقة = 0.9883
الجيل 2: أفضل دقة = 0.9942
الجيل 3: أفضل دقة = 0.9942
الجيل 4: أفضل دقة = 0.9942
الجيل 5: أفضل دقة = 0.9942
الجيل 6: أفضل دقة = 0.9942
الجيل 7: أفضل دقة = 0.9942
الجيل 8: أفضل دقة = 0.9942
الجيل 9: أفضل دقة = 0.9942
الجيل 10: أفضل دقة = 0.9942
الجيل 13: أفضل دقة = 0.9942
الجيل 14: أفضل دقة = 0.9942
الجيل 15: أفضل دقة = 0.9942
الجيل 13: أفضل دقة = 0.9942
الجيل 14: أفضل دقة = 0.9942
الجيل 13: أفضل دقة = 0.9942
الجيل 13: أفضل دقة = 0.9942
الجيل 14: أفضل دقة = 0.9942
الجيل 15: أفضل دقة = 0.9942
الجيل 16: أفضل دقة = 0.9942
الجيل 17: أفضل دقة = 0.9942
الجيل 18: أفضل دقة = 0.9942
الجيل 19: أفضل دقة = 0.9942
الجيل 20: أفضل دقة = 0.9942

✅ أفضل مجموعة خصائص مختارة :
[0, 1, 2, 4, 7, 9, 13, 19, 20, 23, 24, 25, 27, 29]
دقة النموذج النهائي = 0.9942

```